

CatalogGuard Lite

면접 대비 최종 자료

Python·FastAPI 백엔드 프로젝트 기술면접 대비

항목	현재 기준
프로젝트	CatalogGuard Lite
기준일	2026 년 8 월 2 일
기준 HEAD	edd73ed2c00f04aa7d34c0420865463253471dcb
GitHub Actions	Run 30737512739 · test / browser-e2e success
최근 전체 pytest	1042 passed · 88 skipped · 4 deselected
브라우저 E2E	Chromium 2 개 시나리오 · Promotion 포함
AWS Runtime CI	build → import → UID 10001 → migration → Uvicorn → /health

박수용

2026 년 8 월

0. 이 자료를 사용하는 방법

문장을 외우기보다 “문제 → 선택 이유 → 구현 → 검증 → 한계” 흐름을 말할 수 있도록 구성했습니다.

면접 답변 기본 공식

결론을 먼저 말하고, 이유를 1~2 개 설명한 뒤 CatalogGuard Lite 의 실제 구현 근거를 붙입니다. 모르는 부분은 추측하지 않고 “현재는 여기까지 검증했습니다”라고 범위를 구분합니다.

- ★★★★★는 면접 전날 반드시 다시 볼 질문입니다.
- ★★★는 자주 이어질 수 있는 기술 질문입니다.
- ★는 시간이 있을 때 추가로 준비할 질문입니다.
- 테스트 개수보다 실제 결함을 재현하고 회귀 테스트로 막은 경험을 강조합니다.
- 최근 AWS Runtime CI 는 실제 AWS 재배포가 아니라 GitHub Actions Ubuntu runner 의 Docker smoke 입니다.
- 과거 AWS EC2·RDS staging 배포 경험과 최신 CI runtime 검증은 반드시 구분합니다.

프로젝트 한눈에 보기

구분	핵심
문제	공급사마다 다른 CSV 구조·표기와 상품 데이터 오류를 사람이 반복 검수하는 문제
백엔드	FastAPI + Service/Query 계층 + PostgreSQL
비동기	Redis broker/작업 상태 + Celery Worker
ETL	JSON Profile + 공통 Pipeline → normal/reject/summary → staging
Promotion	Preview → 승인 → expected hash 재검증 → 운영 상품 → Run/Audit/History
검증	pytest, PostgreSQL 통합, Async E2E, Streamlit smoke, Chromium E2E, AWS Docker runtime smoke
의도적 제외	인증·권한·rate limit, streaming·증분 ETL, 운영 Redis·Celery 배포 자동화

면접에서 자주 나오는 용어 미니 사전

용어	쉬운 뜻	프로젝트 연결
Hash	내용을 짧은 고정 길이 식별값으로 바꾸는 값	CSV SHA-256 으로 동일 파일 식별
Transaction	여러 DB 변경을 한 묶음으로 처리	Promotion/ETL 중간 실패 시 전체 rollback
Rollback	트랜잭션 실패 시 변경 전으로 되돌림	일부 상품만 반영되는 상태 방지
Staging	운영 반영 전 임시 적재 영역	ETL 정상 상품을 먼저 저장
Audit	무엇이 어떻게 바뀌었는지 남기는 변경 장부	Promotion 상품 이전/새 값 추적
Broker	작업을 Worker 에게 전달하는 중간 전달자	Redis DB 0 → Celery Worker
Worker	큐의 작업을 실제로 실행하는 프로세스	CSV 검수를 백그라운드에서 수행
Migration	DB 스키마 변경을 버전으로 관리	Alembic upgrade head
Runtime	프로그램이 실제 실행되는 시점	Docker build 후 Uvicorn 실행까지 검증
Smoke Test	핵심이 최소한 실제로 뜨는지 빠르게 확인	Streamlit health / AWS image /health
N+1	목록 1 번 후 항목마다 추가 쿼리가 반복되는 문제	Promotion 목록과 Audit 조회 분리
Pagination	많은 결과를 페이지 단위로 나눠 조회	SQL LIMIT/OFFSET 사용

1. 프로젝트 소개 - 30 초·1 분·3 분

30 초 소개

면접 답변

CatalogGuard Lite 는 서로 다른 합성 공급사 CSV 를 JSON 프로파일로 표준화하고 누락·중복·가격·카테고리·개인정보 같은 품질 문제를 검수하는 데이터 품질 백엔드 프로젝트입니다. FastAPI·PostgreSQL 로 검수 이력과 ETL staging 을 관리하고 Redis·Celery 비동기 검수, Preview Hash 기반 Catalog Promotion 과 Audit 을 구현했습니다. GitHub Actions 에서 DB·Redis·비동기 E2E·Chromium E2E 와 AWS Docker runtime 까지 자동 검증해 실제 결함을 회귀 테스트로 남겼습니다.

1 분 소개

쇼핑몰 상품 카탈로그는 공급사마다 컬럼명과 표기 방식이 달라 같은 데이터가 여러 형태로 들어올 수 있습니다. 저는 이 문제를 줄이기 위해 CSV 검수 규칙과 공급사별 JSON Profile 기반 ETL 을 구현했습니다. FastAPI 를 검수의 단일 기준으로 두고 PostgreSQL 에 실행·결과·ETL 배치·운영 상품 이력을 저장했습니다. 시간이 걸리는 검수는 Redis 와 Celery 로 분리했고, staging 상품을 운영 카탈로그에 반영할 때는 Preview 와 사용자 승인, expected hash 재검증, transaction, Audit 을 거치도록 했습니다. 테스트에서는 단위·DB 통합뿐 아니라 실제 Chromium E2E 와 Docker runtime smoke 까지 구성해 과거에 발생한 중복 검수, stale preview, Docker package 누락 같은 문제의 재발을 막는 데 집중했습니다.

3 분 소개

프로젝트의 시작점은 “공급사마다 상품 CSV 구조와 표기가 달라 사람이 반복 확인해야 한다”는 문제였습니다. 먼저 필수값, 형식, 중복, 가격 관계, 상품명·카테고리 불일치, 금지어·개인정보 의심, 색상·사이즈 표준화, 그룹 단위 관계 오류를 규칙 기반으로 검사하도록 만들었습니다. 검수 결과 형식을 공통 ValidationIssue 로 통일하고 FastAPI 를 검수의 단일 기준으로 두어 Streamlit 에서 같은 규칙을 다시 실행하지 않도록 했습니다.

공급사별 CSV 구조 차이는 Python 코드를 공급사마다 복제하지 않고 JSON Profile 에 컬럼 매핑과 기본값을 정의하는 방식으로 분리했습니다. 공통 ETL 은 normal/reject/summary 를 만들고 검증된 상품만 PostgreSQL staging 에 transaction 으로 적재합니다. 이후 staging 데이터를 바로 운영 상품으로 넣지 않고 Promotion Preview 에서 insert/update/no-change 를 먼저 계산하고 사용자가 승인한 뒤 expected_preview_hash 를 다시 검증해 stale 상태를 차단합니다. 성공한 반영은 Promotion Run 과 상품별 Audit 으로 추적합니다.

운영 관점에서는 Redis·Celery 로 긴 검수를 HTTP 요청 수명에서 분리했고 /health 와 /ready, request ID, 구조화 로그를 적용했습니다. 가장 기억에 남는 문제는 AWS Docker image 가 build 는 성공했지만 services 와 workers 가 빠져 실제 실행 시 import 오류가 난 사례입니다. Dockerfile 을 수정한 뒤 GitHub Actions 에서 image build, 핵심 import, non-root UID 10001, migration, 기본 CMD Uvicorn, /health 200 까지 검증하게 만들어 같은 오류가 다시 생기면 CI 에서 바로 실패하도록 했습니다. 현재는 인증·권한·rate limit, streaming ETL, 운영 Redis·Celery 배포 자동화는 MVP 범위에서 제외했고, 지원을 시작하면서 실제 공고 요구에 따라 보완할 계획입니다.

40~60 초 자기소개

자기소개 초안

Python·FastAPI 백엔드 개발을 중심으로 준비하고 있는 박수용입니다. 개인 프로젝트 CatalogGuard Lite 에서 상품 CSV 품질 검수부터 PostgreSQL 저장·조회, Redis·Celery 비동기 처리, 공급사 Profile ETL, Preview 승인 기반 운영 반영과 Audit 까지 직접 구현했습니다. 특히 기능을 많이 붙이는 것보다 실제 오류를 재현하고 테스트와 CI 로 재발을 막는 과정을 중요하게 생각합니다. Docker image 가 build 는 되지만 실제 실행되지 않던 문제를 발견한 뒤 runtime smoke 를 CI 에 추가한 경험처럼, 원인을 확인하고 검증 가능한 형태로 개선하는 백엔드 개발자가 되고 싶습니다.

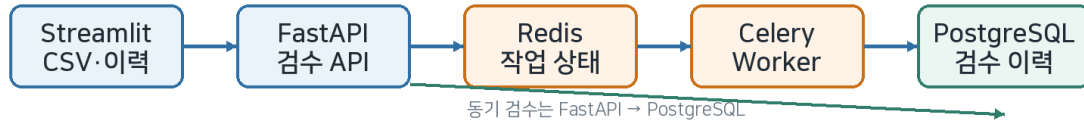
2. 시스템 구조

Streamlit 이 DB 를 직접 수정하지 않고 API Client → FastAPI → Service/Query → PostgreSQL 흐름을 사용합니다.

CatalogGuard Lite 전체 구조

검수 흐름과 ETL·Promotion 흐름을 분리하고 PostgreSQL을 공통 저장 기준으로 사용

동기·비동기 검수



공급사 ETL → staging → 안전한 Promotion



UI는 DB에 직접 쓰지 않고 CatalogGuardApiClient → FastAPI → Service/Query 계층을 거칩니다.

한 문장으로 구조 설명

동기·비동기 검수와 ETL·Promotion 흐름을 분리하되 PostgreSQL 을 공통 저장 기준으로 사용하고, 쓰기 트랜잭션은 Service 에, 읽기 전용 pagination/count 는 Query Service 에 두었습니다.

3. 프로젝트·FastAPI 핵심 질문

Q1. CatalogGuard Lite 는 어떤 프로젝트인가요? ★★★★★

핵심 답변 패션·이커머스 상품 카탈로그의 데이터 품질 오류를 자동으로 검수하고, 공급사별 CSV 를 공통 형식으로 표준화한 뒤 검증된 데이터를 staging 과 운영 카탈로그까지 안전하게 반영하는 Python·FastAPI 백엔드 MVP 입니다.

쉽게 말하면 상품 엑셀을 사람이 하나씩 확인하던 일을 서버가 규칙대로 검사하고, 바로 운영에 넣지 않고 한 번 더 확인한 뒤 반영하는 시스템입니다.

프로젝트 근거 검수 규칙, FastAPI, PostgreSQL, Redis·Celery, Profile ETL, Promotion Preview/Hash/Audit, Streamlit, CI 가 연결되어 있습니다.

추가 질문 대비 면접에서는 “단순 CRUD 가 아니라 데이터 품질과 안전한 반영 흐름을 다룬 프로젝트”라고 먼저 말하면 좋습니다.

Q2. 왜 이 프로젝트를 만들었나요? ★★★★★

핵심 답변 공급사마다 컬럼명과 표기 방식이 달라 같은 상품 데이터가 여러 형태로 들어오고, 누락·중복·가격 관계 오류를 사람이 반복 확인해야 하는 문제를 해결하고 싶었습니다.

쉽게 말하면 같은 검은색이 black, 블랙, BLACK 으로 들어오면 컴퓨터는 서로 다른 값처럼 볼 수 있으므로 공통 기준으로 정리할 필요가 있습니다.

프로젝트 근거 색상·사이즈 정규화, 그룹 카테고리 검수, JSON Profile ETL, reject 분리와 staging 적재로 이 문제를 다뤘습니다.

추가 질문 대비 “실제 기업 데이터 연동은 했나요?”에는 합성 공급사 2 종으로 구조를 검증했고 실연동은 현재 범위에서 제외했다고 답합니다.

Q3. 왜 Python 과 FastAPI 를 선택했나요? ★★★★★

핵심 답변 데이터 처리 로직이 Python 과 pandas 중심이어서 같은 언어로 API 와 검수 로직을 연결하기 쉬웠고, FastAPI 의 Pydantic 검증과 명확한 라우팅 구조가 MVP 에서 API 계약을 관리하기 적합했습니다.

쉽게 말하면 데이터 검사 코드와 웹 API 를 한 언어로 연결해서 구조를 단순하게 유지했습니다.

프로젝트 근거 CSV 업로드·검수·이력·ETL 조회·Promotion API 를 FastAPI router 와 schema/service 계층으로 분리했습니다.

추가 질문 대비 FastAPI 가 항상 Django 보다 좋다는 의미는 아니며 관리자·ORM·인증이 큰 서비스라면 Django 도 좋은 선택이라고 설명합니다.

Q4. Streamlit 에서 직접 검수하지 않고 FastAPI 를 검수의 단일 기준으로 둔 이유는 무엇인가요? ★★★★★

핵심 답변 UI 와 서버가 각각 검수 규칙을 실행하면 결과가 달라질 수 있고 같은 CSV 를 두 번 검사하는 비용도 생깁니다. 그래서 검수는 FastAPI 서비스에서 한 번만 실행하고 Streamlit 은 서버 결과를 표시하도록 통일했습니다.

쉽게 말하면 채점 기준을 두 군데에 두면 서로 다른 답이 나올 수 있으니 서버 한 곳을 정답 기준으로 둔 것입니다.

프로젝트 근거 과거 Streamlit·FastAPI 이중 검수를 제거했고 저장된 검수 결과를 통계·필터·다운로드에 재사용합니다.

추가 질문 대비 이 설계는 UI 를 React 로 바꾸더라도 같은 API 계약을 재사용하기 쉽다는 장점도 있습니다.

Q5. 이 프로젝트에서 가장 중요하게 생각한 설계 원칙은 무엇인가요? ★★★

핵심 답변 기능 수보다 단일 책임, 데이터 무결성, 실패 시 추적 가능성, 그리고 실제 오류를 회귀 테스트로 남기는 것을 중요하게 봤습니다.

쉽게 말하면 “되는 기능”보다 “왜 안전하게 되는지 설명할 수 있는 기능”을 우선했습니다.

프로젝트 근거 Service/Query 분리, transaction/constraint, request ID, /health-/ready, E2E·runtime smoke 가 그 근거입니다.

추가 질문 대비 면접에서는 “복잡하게 만들기보다 실제 문제를 하나씩 검증 가능한 구조로 해결했다”고 정리합니다.

Q6. FastAPI 에서 Service 계층을 분리한 이유는 무엇인가요? ★★★

핵심 답변 라우터가 HTTP 요청 검증과 상태 코드 처리까지 하면서 DB transaction 과 업무 규칙까지 모두 담당하면 테스트와 유지보수가 어려워집니다. 그래서 라우터는 API 계약, Service 는 검수·Promotion 같은 업무 규칙과 쓰기 transaction 을 담당하도록 분리했습니다.

쉽게 말하면 접수 창구와 실제 업무 처리 부서를 나눈 것과 비슷합니다.

프로젝트 근거 Promotion Service 는 transaction 과 hash 재검증을 담당하고 History Query Service 는 읽기 전용 조회를 담당합니다.

추가 질문 대비 모든 작은 함수까지 Service 로 나누는 것이 아니라 책임과 transaction 경계가 명확할 때 분리했다고 설명합니다.

Q7. 같은 CSV 를 다시 업로드하면 어떻게 처리하나요? ★★★★★

핵심 답변 파일 검증·파싱 후 CSV bytes 의 SHA-256 과 inspection_version 으로 기존 검수 이력을 먼저 조회합니다. 같은 파일과 같은 규칙 버전의 결과가 있으면 전체 검수와 신규 저장을 생략하고 기존 결과를 반환합니다.

쉽게 말하면 이미 채점한 같은 시험지는 다시 채점하지 않고 이전 결과를 꺼내는 방식입니다.

프로젝트 근거 API 사전 조회로 불필요한 CPU 작업을 줄이고, 최종 동시 요청은 PostgreSQL partial unique index 로 다시 방어합니다.

추가 질문 대비 파일명은 바뀔 수 있기 때문에 이름이 아니라 내용 기반 hash 를 사용했다고 설명합니다.

Q8. SHA-256 은 왜 사용했나요? ★★★

핵심 답변 파일 내용을 기준으로 동일성을 판별하기 위해 사용했습니다. 파일명이 달라도 bytes 가 같으면 같은 hash 가 나오므로 동일 CSV 를 안정적으로 식별할 수 있습니다.

쉽게 말하면 파일의 지문을 만드는 것입니다.

프로젝트 근거 검수 재사용의 file_sha256 과 ETL 산출물 무결성의 output_file_sha256 에 사용합니다.

추가 질문 대비 SHA-256 을 보안 암호화 자체로 사용한 것이 아니라 동일성·무결성 식별 목적으로 사용했습니다.

Q9. inspection_version 은 왜 필요한가요? ★★★

핵심 답변 검수 규칙이 바뀌면 같은 파일도 새 기준으로 다시 검사해야 하기 때문입니다. hash 만 보면 파일은 같지만 규칙이 달라졌다는 사실을 구분할 수 없습니다.

쉽게 말하면 시험지는 같아도 채점 기준이 바뀌면 다시 채점해야 하는 것과 같습니다.

프로젝트 근거 sale_price 규칙 추가 시 inspection_version 을 5 로 올려 과거 결과가 잘못 재사용되지 않도록 했습니다.

추가 질문 대비 버전 번호는 기능 홍보용 숫자가 아니라 결과 재사용의 정확성을 보장하는 키라고 설명합니다.

Q10. API 에서 요청과 응답 schema 를 명확하게 나눈 이유는 무엇인가요? ★★★

핵심 답변 클라이언트와 서버 사이의 계약을 명확히 하고 잘못된 입력과 예상하지 못한 응답을 조기에 발견하기 위해서입니다.

쉽게 말하면 서로 주고받는 택배 상자의 규격을 미리 정한 것입니다.

프로젝트 근거 FastAPI/Pydantic schema 와 CatalogGuardApiClient 의 응답 schema·SHA-256 형식 검증을 사용합니다.

추가 질문 대비 UI 가 DB 모델을 직접 의존하지 않도록 API 계약을 중간 경계로 둔 점도 설명할 수 있습니다.

Q11. 404, 409 같은 HTTP 상태 코드를 어떻게 사용했나요? ★★★

핵심 답변 없는 ETL/Promotion 실행을 조회하면 404 로 자원이 없음을 표현하고, Preview 시점과 실제 반영 시점 상태가 충돌하는 preview_stale 은 409 Conflict 로 처리합니다.

쉽게 말하면 404 는 “그 데이터가 없음”, 409 는 “요청 형식은 맞지만 현재 상태와 충돌함”에 가깝습니다.

프로젝트 근거 ETLLoadNotFoundError 와 Promotion 오류를 API 계층에서 상태 코드와 도메인 오류로 매핑했습니다.

추가 질문 대비 HTTP 코드는 숫자 자체보다 클라이언트가 실패 원인을 구분해 안전한 안내를 할 수 있게 하는 계약이라고 설명합니다.

Q12. /health 와 /ready 차이는 무엇인가요? ★★★

핵심 답변 /health 는 FastAPI 프로세스가 살아 있는지 확인하고, /ready 는 PostgreSQL 에 SELECT 1 을 실행해 실제 요청을 받을 준비가 됐는지 확인합니다.

쉽게 말하면 가게 불이 켜져 있는지와, 실제로 주문을 받을 준비가 됐는지를 나눈 것입니다.

프로젝트 근거 AWS Docker runtime smoke 는 /health 를 확인하고, 서비스 검증에서는 /ready 로 DB 연결까지 확인했습니다.

추가 질문 대비 DB 가 꺼져도 프로세스는 살아 있을 수 있으므로 두 상태를 분리했다고 설명합니다.

Q13. 구조화 로그와 request ID 는 왜 필요한가요? ★★★

핵심 답변 오류가 발생했을 때 여러 로그 중 어떤 요청에서 문제가 생겼는지 추적하기 위해 사용합니다. 구조화 로그는 필드를 일정한 형식으로 남겨 검색과 분석을 쉽게 합니다.

쉽게 말하면 고객 문의 번호처럼 각 요청에 추적 번호를 붙이는 것입니다.

프로젝트 근거 API 오류와 UI 안내에서 request ID 를 연결하고 readiness·요청 로그를 검증했습니다.

추가 질문 대비 대규모 observability 플랫폼을 구축한 것은 아니며 기본적인 추적 가능성을 확보한 수준이라고 말합니다.

4. PostgreSQL·SQL 질문

Q14. PostgreSQL 을 선택한 이유는 무엇인가요? ★★★★★

핵심 답변 검수 이력, ETL batch, staging 상품, 운영 상품, Promotion Run/Audit 처럼 관계와 제약조건이 중요한 데이터가 많았기 때문에 관계형 DB 가 적합했습니다. PostgreSQL 은 transaction, CHECK, FK, unique/index 와 실제 SQL 실행 계획 검증을 활용하기 좋았습니다.

쉽게 말하면 데이터끼리 연결 관계가 많고 “잘못 저장되지 않게” DB 가 지켜줘야 해서 선택했습니다.

프로젝트 근거 SQLAlchemy·Alembic 과 함께 constraint, transaction, pagination, benchmark 를 사용했습니다.

추가 질문 대비 작은 단일 파일 도구라면 SQLite 가 더 단순할 수 있다고 인정합니다.

Q15. Primary Key 와 Foreign Key 는 프로젝트에서 어디에 사용했나요? ★★★

핵심 답변 Primary Key 는 각 실행·상품·Audit 행을 고유하게 식별하고, Foreign Key 는 ETL batch 와 staging 상품, Promotion Run 과 Audit 처럼 부모·자식 관계를 DB 가 보장하도록 사용합니다.

쉽게 말하면 Primary Key 는 주민번호 같은 고유 번호, Foreign Key 는 “이 상품이 어느 배치에 속하는가”를 연결하는 번호입니다.

프로젝트 근거 etl_load_runs 1 개에 catalog_products_staging 여러 행이 연결되고 DB cascade 동작도 검증했습니다.

추가 질문 대비 FK 는 애플리케이션 코드 실수로 존재하지 않는 부모를 참조하는 것을 막아줍니다.

Q16. partial unique index 가 필요한 이유는 무엇인가요? ★★★★★

핵심 답변 특정 조건을 만족하는 행에만 유일성 규칙을 적용해 동일 파일·버전의 중복 저장을 DB 수준에서 차단하기 위해 사용했습니다. 애플리케이션 조회 이후 동시에 들어온 경쟁 요청까지 막는 최종 안전장치입니다.

쉽게 말하면 모든 데이터가 아니라 “중복을 막아야 하는 조건”에만 번호표 중복 금지를 거는 것입니다.

프로젝트 근거 file_sha256 + inspection_version 조합의 중복 방위에 사용했고 경쟁 요청 테스트로 검증했습니다.

추가 질문 대비 일반 unique constraint 와 달리 일부 상태/조건에만 적용할 수 있다는 점을 설명합니다.

Q17. 애플리케이션 사전 조회가 있는데 DB unique constraint 도 필요한 이유는 무엇인가요? ★★★★★

핵심 답변 두 요청이 거의 동시에 사전 조회하면 둘 다 “아직 없음”을 볼 수 있습니다. 그래서 사전 조회는 불필요한 검수를 줄이는 최적화이고, DB unique 는 최종 데이터 무결성을 보장하는 역할로 분리했습니다.

쉽게 말하면 두 사람이 동시에 빈 좌석을 봤을 때 마지막 예약 확정은 좌석 시스템이 막아줘야 하는 것과 같습니다.

프로젝트 근거 동일 CSV 재검수 방지는 API 사전 조회, 경쟁 저장 중복은 DB partial unique index 가 담당합니다.

추가 질문 대비 이 상황을 race condition, 즉 실행 순서에 따라 결과가 달라지는 경쟁 상태라고 설명할 수 있습니다.

Q18. Transaction 과 Rollback 은 왜 필요한가요? ★★★★★

핵심 답변 여러 DB 변경 중 하나가 실패했을 때 일부만 남으면 데이터가 서로 맞지 않게 됩니다. 그래서 ETL batch+상품 저장과 Promotion 운영 반영+Run/Audit 을 transaction 으로 묶고 실패하면 전체 rollback 하도록 했습니다.

쉽게 말하면 송금에서 출금만 되고 입금이 안 되면 안 되는 것과 같습니다.

프로젝트 근거 staging 적재와 Promotion 에서 일부 실패 시 전체 rollback 테스트를 수행했습니다.

추가 질문 대비 transaction boundary 를 Service 에 둔 이유와 연결해서 설명할 수 있습니다.

Q19. JOIN 은 프로젝트에서 왜 필요한가요? ★★★

핵심 답변 Promotion 실행 이력과 관련 ETL 정보처럼 서로 다른 테이블의 정보를 한 화면에서 보여줄 때 관계를 따라 데이터를 결합하기 위해 사용합니다.

쉽게 말하면 서로 다른 명단을 공통 ID 로 맞춰 한 줄에 보여주는 것입니다.

프로젝트 근거 Catalog Promotion history 에서 ETL run 과 실행 정보를 연결해 조회합니다.

추가 질문 대비 목록에서 불필요하게 Audit 전체까지 JOIN 하지 않고 필요한 시점에 별도 조회해 응답 크기와 N+1 을 관리했습니다.

Q20. N+1 문제가 무엇이고 이 프로젝트에서는 어떻게 피했나요? ★★★

핵심 답변 목록을 한 번 조회한 뒤 각 항목마다 추가 쿼리를 반복해 총 N+1 번 이상 DB 를 호출하는 문제입니다. Promotion 목록에서는 Audit 전체를 항목별로 자동 로딩하지 않고 목록은 Run 정보만, 선택한 실행의 Audit 만 별도로 조회합니다.

쉽게 말하면 20 명의 명단을 가져온 뒤 한 명씩 다시 DB 에 전화하는 일을 피한 것입니다.

프로젝트 근거 History Query Service 에서 목록·count 와 상세/Audit 책임을 분리했고 조회 전후 DB 불변도 확인했습니다.

추가 질문 대비 모든 “여러 쿼리”가 N+1 은 아니며 bounded query 처럼 개수가 고정된 조회는 구분해야 한다고 답합니다.

Q21. LIMIT / OFFSET pagination 을 사용한 이유는 무엇인가요? ★★★

핵심 답변 현재 MVP 데이터 규모와 UI 요구에서는 구현이 단순하고 페이지 번호를 제공하기 쉬워 SQL LIMIT/OFFSET 을 선택했습니다.

쉽게 말하면 전체 목록을 한 번에 들고 오지 않고 필요한 구간만 잘라 받는 방식입니다.

프로젝트 근거 ETL 목록, staging 상품, Promotion history 에서 서버 SQL pagination 과 count 를 사용합니다.

추가 질문 대비 데이터가 매우 커지고 뒤 페이지 탐색 비용이 문제가 되면 keyset/cursor pagination 을 검토할 수 있습니다.

Q22. 왜 cursor pagination 으로 바꾸지 않았나요? ★★★

핵심 답변 현재 측정된 규모에서 OFFSET 이 병목이라는 근거가 없고, cursor 는 정렬 키·UI 상태·API 계약 복잡도가 올라갑니다. 측정 없이 구조를 복잡하게 바꾸지 않는 원칙을 적용했습니다.

쉽게 말하면 지금 계단이 충분한데 이용자가 많지 않다고 바로 엘리베이터 공사를 하지 않은 것입니다.

프로젝트 근거 기존 SQL benchmark 에서 Inspection 주요 조회에 병목 근거가 없었고 ETL/Promotion history 는 향후 측정 후보로 남겼습니다.

추가 질문 대비 “필요하면 바꿀 수 있지만 지금은 근거가 없다”가 핵심입니다.

Q23. COUNT 쿼리는 왜 별도로 사용했나요? ★★★

핵심 답변 페이지 목록과 전체 페이지 수를 함께 제공하려면 현재 필터에 해당하는 전체 개수가 필요합니다. 목록 데이터와 같은 필터 함수를 count 에도 적용해 페이지 수 불일치를 막았습니다.

쉽게 말하면 현재 10 개만 보여줘도 전체가 73 개인지 알아야 “8 페이지”를 계산할 수 있습니다.

프로젝트 근거 ETL 목록과 Promotion history 에서 목록 쿼리와 count 를 분리합니다.

추가 질문 대비 count 가 항상 빠른 것은 아니므로 데이터가 커지면 별도 최적화가 필요할 수 있습니다.

Q24. Seq Scan 이 무엇인가요? ★★★

핵심 답변 PostgreSQL 이 테이블을 처음부터 끝까지 읽는 실행 계획입니다. 무조건 나쁜 것이 아니라 테이블이 작거나 대부분 행을 읽어야 할 때는 인덱스보다 합리적일 수 있습니다.

쉽게 말하면 책에서 거의 모든 페이지를 볼 거면 목차를 계속 찾는 것보다 처음부터 읽는 게 빠를 수 있습니다.

프로젝트 근거 기존 10,000 행 count(*) benchmark 는 Seq Scan 이었지만 중앙값 약 0.872 ms 라 병목 근거가 없었습니다.

추가 질문 대비 실행 계획은 이름만 보고 좋고 나쁨을 결정하지 않는다고 말합니다.

Q25. Index Scan 은 무엇인가요? ★★★

핵심 답변 인덱스를 먼저 찾아 필요한 행 위치로 접근하는 실행 계획입니다. 조건이 선택적이고 인덱스가 쿼리와 잘 맞을 때 유리합니다.

쉽게 말하면 책 뒤의 색인에서 단어 위치를 찾고 바로 해당 페이지로 가는 방식입니다.

프로젝트 근거 동일 CSV 조회와 검수 상세 등 기존 benchmark 에서 Index Scan 이 사용되었습니다.

추가 질문 대비 인덱스는 조회를 빠르게 할 수 있지만 쓰기·저장 공간 비용이 있으므로 무조건 추가하지 않습니다.

Q26. PostgreSQL SQL 성능은 어떻게 확인했나요? ★★★★★

핵심 답변 전용 performance 테스트에서 10,000 inspection_runs 와 100,000 inspection_results 합성 데이터를 넣고 EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON)을 사용했습니다. warmup 2 회 후 7 회 측정해 중앙값을 기록했습니다.

쉽게 말하면 DB 에게 “이 쿼리를 어떤 순서로 실행했고 실제로 얼마나 걸렸는지” 보고서를 받았습니다.

프로젝트 근거 기존 저장 결과는 동일 CSV 0.018 ms, 목록 0.044 ms, count 0.872 ms, 상세 0.016 ms, 결과 0.029 ms 입니다.

추가 질문 대비 최근 세션에서 새로 측정한 것이 아니라 기존 PostgreSQL 18 benchmark 를 재검토했던 결과라고 정확히 구분합니다.

Q27. 왜 새로운 index 를 추가하지 않았나요? ★★★★★

핵심 답변 현재 측정 범위에서 병목 근거가 없었기 때문입니다. 특히 count(*) Seq Scan 도 10,000 건에서 약 0.872 ms 였고, 인덱스는 쓰기 비용과 저장 공간을 늘리므로 근거 없이 추가하지 않았습니다.

쉽게 말하면 “인덱스가 많을수록 좋다”가 아니라 실제 느린 구간이 있을 때 필요한 인덱스를 추가하는 방식입니다.

프로젝트 근거 성능 분석 최종 판정은 현재 측정 범위에서 수정 불필요였고 migration/index 변경을 하지 않았습니다.

추가 질문 대비 ETL/Promotion history 는 fresh benchmark 가 없어 향후 실제 데이터가 쌓이면 별도 측정 후보입니다.

5. Redis·Celery 비동기 질문

Q28. Redis 와 Celery 를 왜 사용했나요? ★★★★★

핵심 답변 CSV 검수처럼 시간이 늘어날 수 있는 작업을 HTTP 요청 안에서 끝까지 기다리게 하지 않고 백그라운드 Worker 로 분리하기 위해 사용했습니다. Redis 는 작업 전달과 상태 저장, Celery Worker 는 실제 검수를 담당합니다.

쉽게 말하면 접수 창구가 일을 직접 끝낼 때까지 손님을 세워두지 않고 작업표를 작업자에게 넘기는 구조입니다.

프로젝트 근거 FastAPI → Redis → Celery Worker → PostgreSQL 흐름을 Async E2E 로 검증했습니다.

추가 질문 대비 작업이 매우 짧고 단순하면 동기 처리나 BackgroundTasks 가 더 단순할 수 있다고 인정합니다.

Q29. Celery 작업 상태는 어떻게 관리하나요? ★★★

핵심 답변 queued, running, succeeded, failed 상태를 프로젝트 자체 job 상태로 관리하고 클라이언트가 명시적으로 상태를 조회합니다.

쉽게 말하면 접수됨 → 처리 중 → 성공/실패의 네 단계로 보는 것입니다.

프로젝트 근거 Redis DB 1 에 job 상태를 두고 완료 결과는 PostgreSQL 에 저장합니다.

추가 질문 대비 상태와 실제 결과 저장소를 분리해 Celery backend 에 종속되지 않도록 했다고 설명합니다.

Q30. Celery 작업 실패 시 어떻게 처리하나요? ★★★

핵심 답변 failed 상태를 사용자에게 표시하고 같은 요청을 자동으로 무한 재시도하지 않습니다. 사용자가 원인을 확인한 뒤 명시적으로 재제출할 수 있게 했고 임시 CSV 파일은 성공·실패 후 정리합니다.

쉽게 말하면 실패했다고 몰래 계속 반복하지 않고, 실패 사실을 보여준 뒤 사용자가 다시 실행하도록 합니다.

프로젝트 근거 failed 후 재제출, 중복 POST 방지, 임시 파일 cleanup 을 UI/Async E2E 에서 검증했습니다.

추가 질문 대비 운영 환경의 자동 retry 정책과 모니터링은 현재 MVP 범위에 포함하지 않았습니다.

Q31. 왜 Celery result backend 를 사용하지 않았나요? ★★★

핵심 답변 task_ignore_result 설정으로 Celery 자체 결과 저장을 사용하지 않고, 사용자에게 필요한 job 상태는 Redis DB 1, 영속 결과는 PostgreSQL 에 저장하도록 프로젝트 데이터 모델에 맞췄습니다.

쉽게 말하면 작업 도구가 따로 결과를 보관하게 하지 않고 서비스가 쓰는 저장소에 상태와 결과를 나눠 저장한 것입니다.

프로젝트 근거 Redis DB 0 은 broker, DB 1 은 job 상태, 검수 결과는 PostgreSQL 에 남습니다.

추가 질문 대비 이 선택이 모든 서비스의 정답은 아니며 Celery chaining/결과 조회가 핵심이면 result backend 가 편할 수 있습니다.

Q32. Redis 에서 broker 와 job 상태를 분리한 이유는 무엇인가요? ★★★

핵심 답변 작업 전달용 큐 데이터와 사용자에게 보여줄 job 상태의 책임을 구분해 관리하기 위해 DB 번호를 나눴습니다.

쉽게 말하면 우편함과 작업 현황판을 같은 Redis 서버 안에서 다른 칸으로 나눈 것입니다.

프로젝트 근거 Redis DB 0 은 Celery broker, DB 1 은 inspection job 상태로 사용합니다.

추가 질문 대비 보안 경계 자체가 완전히 분리되는 것은 아니며 논리적인 분리라는 점은 구분합니다.

Q33. FastAPI BackgroundTasks 대신 Celery 를 선택한 이유는 무엇인가요? ★★★

핵심 답변 BackgroundTasks 는 같은 애플리케이션 프로세스의 수명과 자원에 더 가까이 묶여 있고 별도 Worker·queue 기반 상태 관리가 필요한 경우에는 Celery 가 더 적합하다고 판단했습니다.

쉽게 말하면 간단한 뒷정리는 서버가 해도 되지만, 긴 일을 별도 작업자에게 맡기려면 큐가 더 적합한 것입니다.

프로젝트 근거 Redis broker 와 별도 Celery Worker, 상태 polling, 실패·재제출 흐름을 실제로 구현했습니다.

추가 질문 대비 이 MVP 의 실제 처리 시간이 항상 매우 길다는 의미가 아니라 확장 가능한 비동기 실행 흐름을 검증한 것입니다.

Q34. 동기 처리와 비동기 Worker 방식의 장단점은 무엇인가요? ★

핵심 답변 동기는 구현과 디버깅이 단순하지만 요청이 끝날 때까지 사용자가 기다려야 합니다. Worker 방식은 요청을 빠르게 반환하고 작업을 분리할 수 있지만 queue·상태·실패·재시도 관리가 추가됩니다.

쉽게 말하면 바로 처리하는 창구와 번호표를 뽑고 나중에 결과를 확인하는 창구의 차이입니다.

프로젝트 근거 CatalogGuard Lite 는 동기 API 와 비동기 inspection-jobs 를 모두 두어 두 흐름을 비교할 수 있습니다.

추가 질문 대비 무조건 비동기가 좋은 것이 아니라 작업 시간과 운영 복잡도를 보고 선택한다고 답합니다.

6. ETL 질문

Q35. ETL 이 무엇인가요? ★★★★★

핵심 답변 Extract, Transform, Load 의 약자로 데이터를 읽고, 필요한 형태로 변환한 뒤, 저장소에 적재하는 과정입니다. CatalogGuard Lite 에서는 공급사 CSV 를 읽어 공통 상품 schema 로 변환하고 검증된 상품을 PostgreSQL staging 에 저장합니다.

쉽게 말하면 서로 다른 양식의 주문서를 읽어 회사 표준 양식으로 바꾼 다음 창고 시스템에 넣는 과정입니다.

프로젝트 근거 공급사 CSV → JSON Profile → 공통 ETL → normal/reject/summary → staging 흐름입니다.

추가 질문 대비 ETL 은 단순 파일 복사가 아니라 변환·검증·적재 이력을 포함하는 데이터 처리 파이프라인이라고 설명합니다.

Q36. 공급사마다 CSV 형식이 다른 문제를 어떻게 해결했나요? ★★★★★

핵심 답변 공급사별 컬럼 이름과 기본값 규칙을 JSON Profile 로 분리하고 공통 transformer/pipeline 이 그 프로필을 읽어 표준 컬럼으로 변환하도록 했습니다.

쉽게 말하면 공급사마다 새 프로그램을 만드는 대신 “이 공급사의 컬럼은 이렇게 바뀌라”라는 설정표만 추가하는 방식입니다.

프로젝트 근거 vendor_sku, style_id, sku_code, discount_price, promo_price 가 product_group_id, product_id, sale_price 같은 공통 필드로 매핑됩니다.

추가 질문 대비 두 번째 프로필 추가 시 profile_loader.py, transformer.py, pipeline.py 를 수정하지 않은 점을 강조합니다.

Q37. 왜 공급사마다 Python 코드를 만들지 않고 JSON Profile 을 사용했나요? ★★★★★

핵심 답변 공급사가 늘어날 때 if/else 와 공급사 전용 코드를 계속 추가하면 공통 로직이 중복되고 수정 영향 범위가 커집니다. 데이터 구조 차이는 설정으로 분리하고 변환 알고리즘은 공통 코드로 유지하려고 했습니다.

쉽게 말하면 요리법은 하나로 두고 재료표만 공급사별로 바꾸는 방식입니다.

프로젝트 근거 합성 공급사 2 종을 같은 공통 ETL 코드로 처리했습니다.

추가 질문 대비 Profile 자체가 복잡해질 수 있으므로 모든 특수 규칙을 무조건 JSON 에 넣기보다 공통으로 표현 가능한 차이에 사용한다고 답합니다.

Q38. reject CSV 는 왜 필요한가요? ★★★

핵심 답변 변환 또는 검증에 실패한 행을 정상 데이터와 섞지 않고 분리해 운영자가 어떤 데이터가 왜 거부됐는지 확인할 수 있게 하기 위해서입니다.

쉽게 말하면 불량품을 버리는 대신 별도 상자에 담아 이유를 확인할 수 있게 한 것입니다.

프로젝트 근거 ETL 은 normal CSV, reject CSV, summary JSON 을 생성하고 개인정보 의심 정보는 안전하게 마스킹합니다.

추가 질문 대비 현재 reject 결과를 별도 DB 에 영속 저장하는 기능은 범위에 포함하지 않았다고 구분합니다.

Q39. staging 테이블을 따로 둔 이유는 무엇인가요? ★★★★★

핵심 답변 외부에서 들어온 변환 데이터를 바로 운영 상품에 쓰지 않고, 검증·조회·Preview 를 거칠 수 있는 중간 안전 구역이 필요했기 때문입니다.

쉽게 말하면 매장 진열 전에 창고의 검수 구역에 먼저 두는 것입니다.

프로젝트 근거 catalog_products_staging 과 etl_load_runs 에 batch 단위로 저장한 뒤 Promotion Preview 를 수행합니다.

추가 질문 대비 staging 을 두면 운영 반영과 데이터 수집/변환 책임을 분리할 수 있습니다.

Q40. ETL 적재에서 transaction 은 왜 필요한가요? ★★★★★

핵심 답변 배치 기록은 만들어졌는데 상품 일부만 저장되는 상태를 막기 위해 batch 생성과 상품 저장을 하나의 transaction 으로 묶었습니다.

쉽게 말하면 송장만 만들어지고 물건은 절반만 들어오는 상황을 막는 것입니다.

프로젝트 근거 일부 실패 시 전체 rollback, FK·CHECK·cascade 를 PostgreSQL 통합 테스트로 검증했습니다.

추가 질문 대비 DB 제약과 애플리케이션 검증을 함께 사용하는 이유도 연결해서 말할 수 있습니다.

Q41. ETL 산출물 무결성은 어떻게 확인하나요? ★★★

핵심 답변 표준 CSV 의 SHA-256 과 summary JSON 의 output_file_sha256, 실제 행 수와 loaded_rows 가 모두 일치할 때만 staging 에 적재합니다.

쉽게 말하면 파일 내용과 요약표가 정말 같은 결과를 가리키는지 다시 대조하는 것입니다.

프로젝트 근거 파일 hash 와 행 수를 loader 에서 검증한 뒤 transaction 적재합니다.

추가 질문 대비 이 검증은 전송 중/파일 교체/잘못된 summary 같은 오류를 조기에 발견하는 데 도움이 됩니다.

Q42. ETL 실행을 웹 API 로 만들지 않은 이유는 무엇인가요? ★

핵심 답변 현재 MVP 에서는 공급사 변환·적재 자체보다 안정적인 공통 ETL 과 저장·조회·Promotion 흐름을 검증하는 데 집중해 실행은 CLI 중심으로 유지했습니다.

쉽게 말하면 자동화 버튼보다 먼저 실제 변환 로직과 DB 무결성을 확인한 것입니다.

프로젝트 근거 Streamlit 은 ETL 이력·Promotion UI 를 제공하지만 ETL 변환 실행 웹 API 는 현재 범위에서 제외되어 있습니다.

추가 질문 대비 실사용 요구가 생기면 인증·권한·파일 업로드 정책과 함께 실행 API 를 설계하는 것이 안전합니다.



7. Promotion·Audit·Docker·CI 핵심 질문

Q43. Promotion 이 무엇인가요? ★★★★★

핵심 답변 이 프로젝트에서 Promotion 은 할인 행사가 아니라 ETL staging 의 검증된 상품을 운영 카탈로그 테이블에 반영하는 과정입니다.

쉽게 말하면 창고의 검수 완료 상품을 실제 매장 진열 상품으로 옮기는 단계입니다.

프로젝트 근거 Preview → 승인 → expected hash → insert/update/no-change → Run/Audit/History 흐름입니다.

추가 질문 대비 면접에서 “promotion=할인”으로 오해하지 않도록 첫 문장에 정의를 먼저 말합니다.

Q44. 왜 staging 데이터를 바로 운영 상품에 넣지 않았나요? ★★★★★

핵심 답변 자동 변환 결과가 정확하더라도 운영 데이터 변경은 영향이 크므로, 사용자가 무엇이 바뀌는지 Preview 로 확인하고 명시적으로 승인하는 단계가 필요하다고 판단했습니다.

쉽게 말하면 검수된 상품이라도 바로 진열하지 않고 마지막 확인을 거치는 것입니다.

프로젝트 근거 confirmed 승인 checkbox 와 Preview 결과가 없으면 실제 Promotion 을 실행할 수 없도록 UI/API 를 구성했습니다.

추가 질문 대비 운영 데이터 변경은 “자동화 편의성”보다 “잘못된 반영 방지와 추적성”을 우선했다고 설명합니다.

Q45. Promotion Preview 는 왜 필요한가요? ★★★★★

핵심 답변 실제 DB 를 변경하기 전에 신규, 수정, 변경 없음 건수와 상품별 변경 전·후 값을 보여줘 사용자가 영향 범위를 확인하게 하기 위해서입니다.

쉽게 말하면 저장 버튼을 누르기 전에 “무엇이 바뀔 예정인지” 미리 보여주는 것입니다.

프로젝트 근거 Preview API 가 insert/update/no-change 와 반영 불가 사유, preview hash 를 반환합니다.

추가 질문 대비 Preview 는 읽기 단계이고 실제 쓰기 transaction 은 Promotion 실행 시점에만 시작합니다.

Q46. expected_preview_hash 는 무엇인가요? ★★★★★

핵심 답변 사용자가 확인한 Preview 내용의 canonical 결과를 SHA-256 으로 만든 식별값입니다. 실제 실행할 때 서버가 현재 Preview 를 다시 계산해 전달받은 hash 와 같은지 확인합니다.

쉽게 말하면 사용자가 승인한 견적서에 도장을 찍고 실행 직전 같은 견적서인지 다시 확인하는 방식입니다.

프로젝트 근거 lowercase SHA-256 형식을 Client 에서도 검증하고, 실행 API 는 confirmed 와 expected_preview_hash 를 받습니다.

추가 질문 대비 hash 는 원본 데이터를 숨기는 보안 장치가 아니라 “승인한 상태와 실행 상태가 같은지” 확인하는 무결성 장치입니다.

Q47. preview_stale 은 어떤 상황인가요? ★★★★★

핵심 답변 Preview 를 본 뒤 staging 이나 운영 상품 상태가 바뀌어 현재 계산 결과가 사용자가 승인한 결과와 달라진 상황입니다. 이때 실행을 막고 409 Conflict 의 preview_stale 오류로 다시 Preview 를 받도록 합니다.

쉽게 말하면 승인한 견적이 바뀌었는데 예전 승인으로 결제하려는 상황을 막는 것입니다.

프로젝트 근거 실행 시 canonical Preview 를 재계산하고 hash 가 다르면 운영 반영을 하지 않습니다.

추가 질문 대비 stale 상태에서 “어차피 비슷하니 실행”하지 않고 사용자의 재확인을 요구하는 것이 핵심입니다.

Q48. Promotion Audit 은 왜 만들었나요? ★★★★★

핵심 답변 운영 상품이 바뀐 뒤 현재 값만 보면 어떤 실행에서 무엇이 변경됐는지 알기 어렵기 때문에 이전 값·새 값·관련 Promotion Run 을 남겨 추적할 수 있게 했습니다.

쉽게 말하면 상품 변경 장부를 남기는 것입니다.

프로젝트 근거 History UI 에서 상품별 Audit 의 변경 전·후 값을 조회할 수 있고 조회 전후 DB 상태 불변도 검증했습니다.

추가 질문 대비 Audit 은 장애 조사뿐 아니라 운영 변경 책임과 설명 가능성을 높이는 역할이라고 말합니다.

Q49. Promotion History 는 왜 필요한가요? ★★★

핵심 답변 과거 어떤 ETL batch 가 언제 어떤 결과로 운영 반영됐는지 사용자가 화면에서 추적할 수 있어야 하기 때문입니다.

쉽게 말하면 오늘 결과만 보는 것이 아니라 과거 작업 기록을 다시 확인하는 기능입니다.

프로젝트 근거 목록·상태 필터·상세·Audit GET API 와 Streamlit UI 를 구현했습니다.

추가 질문 대비 목록에서는 Audit 전체를 미리 읽지 않아 N+1 과 불필요한 응답 크기를 피했습니다.

Q50. Docker build 성공만으로 충분하지 않았던 이유는 무엇인가요? ★★★★★

핵심 답변 이미지는 정상적으로 만들어져도 실행에 필요한 Python package 가 빠질 수 있기 때문입니다. 실제로 Dockerfile.aws build 는 성공했지만 services 와 workers 가 image 에 없어 Uvicorn 시작 시 import 오류가 발생했습니다.

쉽게 말하면 태백 상자는 만들어졌지만 필요한 부품이 빠져 실제 제품이 켜지지 않은 상황입니다.

프로젝트 근거 ModuleNotFoundError: No module named services 를 발견하고 COPY 를 보완한 뒤 runtime 검증을 추가했습니다.

추가 질문 대비 이 경험 때문에 “build 통과”와 “runtime 동작”을 별도 검증해야 한다고 배웠다고 답합니다.

Q51. GitHub Actions 에서 AWS Docker runtime 을 어떻게 검증하나요? ★★★★★

핵심 답변 Dockerfile.aws image 를 build 한 뒤 컨테이너 내부 핵심 import 와 기본 non-root UID 10001 을 확인하고, PostgreSQL migration 을 적용한 뒤 Dockerfile 의 기본 CMD 로 Uvicorn 을 실제 시작해 /health HTTP 200 을 최대 30 초 안에 확인합니다.

쉽게 말하면 상자를 만들고 안의 부품을 확인한 뒤 실제로 전원을 켜서 최소 기능이 살아 있는지 검사합니다.

프로젝트 근거 실패 시 컨테이너 로그를 출력하고 cleanup 하며 Run 30737512739 에서 성공했습니다.

추가 질문 대비 이것은 실제 AWS 재배포가 아니라 GitHub Actions Ubuntu runner 의 AWS image runtime smoke 라는 점을 구분합니다.

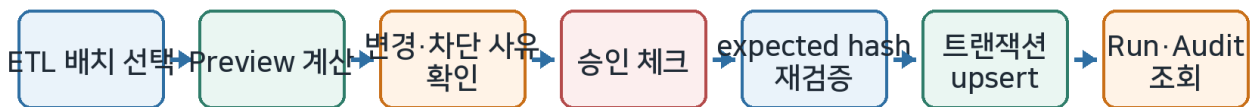
Q52. 현재 프로젝트의 한계와 앞으로 하나만 개선한다면 무엇인가요? ★★★★★

핵심 답변 인증·권한·rate limit, 외부 공급사 실연동, streaming·증분 ETL, 운영 Redis·Celery 배포 자동화, 대규모 동시 트래픽 측정은 현재 범위에 없습니다. 지금은 지원을 시작하고 실제 공고에서 반복 요구되는 부족 기술을 확인한 뒤 가장 필요한 한 가지를 보완하겠습니다.

쉽게 말하면 완벽한 실서비스를 흉내 내기보다 현재 MVP 가 어디까지인지 정확히 구분하고 다음 투자를 채용 요구에 맞추겠다는 의미입니다.

프로젝트 근거 프로젝트 최종 리뷰에서 Blocking 문제 없이 “지금 바로 지원 가능”으로 판단했고 새 기능보다 제출 자료 정합성을 우선했습니다.

추가 질문 대비 과거에는 브라우저 E2E 가 후속 과제였지만 현재는 구현 완료이므로 예전 답변을 그대로 사용하면 안 됩니다.



8. 테스트·CI·성능 추가 질문

Q53. GitHub Actions에서는 어떤 테스트가 실행되나요? ★★★★★

핵심 답변 PostgreSQL 18 과 Redis 7.4 를 올리고 Alembic migration, 일반 pytest, Celery Worker/FastAPI readiness, 비동기 E2E, Streamlit startup smoke 를 수행합니다. 별도 browser-e2e job에서는 Playwright Chromium 시나리오를 실행하고 AWS Docker runtime smoke 도 자동 검증합니다.

쉽게 말하면 GitHub 의 새 컴퓨터에서 DB 와 큐, 서버, 브라우저까지 다시 연결해 보는 것입니다.

프로젝트 근거 최근 Run 30737512739 에서 test 와 browser-e2e 가 모두 성공했습니다.

추가 질문 대비 정확한 테스트 숫자는 실행 조건에 따라 달라질 수 있으므로 구조와 검증 범위를 중심으로 설명합니다.

Q54. 단위 테스트와 E2E 테스트 차이는 무엇인가요? ★★★★★

핵심 답변 단위 테스트는 함수나 규칙처럼 작은 단위를 빠르게 검증하고, E2E 는 실제 사용 흐름처럼 여러 서비스가 연결된 상태에서 처음부터 끝까지 검증합니다.

쉽게 말하면 부품 하나를 시험하는 것과 완성차를 실제 도로에서 주행해 보는 차이입니다.

프로젝트 근거 검수 규칙 단위 테스트와 FastAPI+Redis+Celery+PostgreSQL Async E2E, Streamlit+FastAPI+DB Chromium E2E 를 함께 사용합니다.

추가 질문 대비 E2E 만 많으면 느리고 원인 찾기 어려우므로 단위·통합·E2E 를 역할별로 나눕니다.

Q55. Chromium Browser E2E 까지 만든 이유는 무엇인가요? ★★★★★

핵심 답변 AppTest 와 API 테스트가 모두 통과해도 실제 DOM 구조, checkbox selector, 배치 선택 흐름처럼 브라우저에서만 드러나는 문제가 있기 때문입니다.

쉽게 말하면 각 부품이 정상이어도 실제 사용자가 버튼을 누르는 순서에서 문제가 생길 수 있어 실제 브라우저를 사용했습니다.

프로젝트 근거 ETL → Preview → 승인 → Promotion → History → Audit 흐름과 최종 PostgreSQL 상태를 검증했고 selector/assertion 결함도 실제로 발견했습니다.

추가 질문 대비 브라우저 E2E 2 개 시나리오는 “테스트 숫자”보다 실제 사용자 경로를 자동 재현한다는 점이 중요합니다.

Q56. 테스트가 1,000 개가 넘는데 개수 자체가 중요한가요? ★★★★★

핵심 답변 아닙니다. 개수보다 핵심 기능·경계조건·실제 장애가 다시 발생하지 않는지 검증하는지가 중요합니다. 저는 중복 CSV, stale preview, Docker runtime package 누락처럼 실제로 겪은 문제를 회귀 테스트로 남긴 점을 더 중요하게 봅니다.

쉽게 말하면 시험 문제 수가 많다는 것보다 중요한 내용을 제대로 검사하는지가 더 중요합니다.

프로젝트 근거 최근 전체 pytest 는 1042 passed, 88 skipped, 4 deselected 였지만 면접에서는 숫자보다 테스트 계층과 문제 해결 사례를 설명합니다.

추가 질문 대비 skipped/deselected 도 실패가 아니라 환경·marker 에 따른 의도적 제외가 있을 수 있어 맥락을 설명합니다.

Q57. 가장 기억에 남는 버그는 무엇인가요? ★★★

핵심 답변 AWS Docker image 가 build 는 성공했지만 실제 실행 시 services package 가 없어 서버가 시작되지 않았던 문제입니다. 단 순 build 검증만으로는 잡지 못해 runtime smoke 를 CI 에 추가했다는 점이 가장 기억에 납니다.

쉽게 말하면 “만들어짐”과 “실제로 켜짐”이 다르다는 것을 배운 사례입니다.

프로젝트 근거 Dockerfile.aws packaging 수정 + GitHub Actions import/UID/migration/default CMD/health 검증으로 재발을 막았습니다.

추가 질문 대비 UI 에서는 Promotion History cache 와 hidden checkbox selector 같은 버그도 있었지만 Docker 사례가 문제→재발 방지 흐름을 가장 잘 보여줍니다.

Q58. 프로젝트에서 가장 어려웠던 문제는 무엇인가요? ★★★★★

핵심 답변 기능 하나보다 여러 상태가 엮이는 Promotion 안전성 설계가 가장 어려웠습니다. 사용자가 본 Preview 와 실행 시점 데이터가 달라질 수 있어 hash 재검증, transaction, 승인 상태, Audit, UI cache 까지 함께 맞춰야 했습니다.

쉽게 말하면 “미리 본 결과와 실제 반영 결과가 달라지면 안 된다”는 문제를 여러 계층에서 동시에 해결해야 했습니다.

프로젝트 근거 preview_stale, rollback, concurrent 실행, History/Audit, 브라우저 E2E 로 검증했습니다.

추가 질문 대비 면접관이 더 깊게 물으면 hash 만으로 끝내지 않고 transaction 과 DB 상태 재조회까지 연결해서 설명합니다.

Q59. 다시 만든다면 무엇을 다르게 하겠나요? ★★★

핵심 답변 처음부터 새 기술을 더 많이 넣기보다 현재처럼 기능을 단계적으로 만들되, 핵심 사용자 흐름의 브라우저 E2E 와 운영용 Docker runtime smoke 를 조금 더 일찍 추가했을 것 같습니다. 두 검증이 실제 결함을 잡는 데 효과가 컸기 때문입니다.

쉽게 말하면 기능 개발 마지막에 테스트를 붙이기보다 위험한 흐름부터 실제 실행 검증을 일찍 두겠다는 의미입니다.

프로젝트 근거 Browser E2E 와 AWS runtime CI 는 후반에 추가됐지만 hidden selector 와 package 누락을 실제로 잡았습니다.

추가 질문 대비 반면 Redis·React·laC 같은 기술을 처음부터 모두 넣는 선택은 하지 않을 것이라고 말합니다.

9. 기술 선택 비교 질문

현재 선택을 무조건 최고라고 말하지 않고, 프로젝트 조건에서 왜 맞았는지와 다른 선택이 유리한 경우를 함께 설명합니다.

비교	현재 선택	왜 선택했나	다른 선택이 더 나은 경우
FastAPI vs Django	FastAPI	데이터 처리 Python 코드와 API 를 가볍게 연결하고 Pydantic 기반 계약을 명확히 하기 좋았습니다.	관리자·인증·ORM·템플릿 등 풀스택 기능이 큰 서비스라면 Django 가 더 생산적일 수 있습니다.
PostgreSQL vs SQLite	PostgreSQL	동시 요청, transaction, FK/CHECK/partial unique index 와 실제 SQL 실행 계획을 검증해야 했습니다.	개인 로컬 도구나 단일 프로세스의 작은 MVP 라면 SQLite 가 훨씬 단순할 수 있습니다.
Redis/Celery vs BackgroundTasks	Redis/Celery	별도 Worker·queue·명시적 job 상태·실패/재제출 흐름을 검증하고 싶었습니다.	짧은 이메일 전송·로그 정리처럼 프로세스 내 간단한 후처리는 BackgroundTasks 가 더 단순합니다.
Streamlit vs React	Streamlit	백엔드 데이터 흐름을 빠르게 보여주고 Python 중심으로 MVP 를 검증하기 좋았습니다.	복잡한 상호작용·디자인 시스템·대규모 프론트 상태가 중요하다면 React 가 적합합니다.
JSON Profile vs 공급사별 Python	JSON Profile	공통으로 표현 가능한 컬럼 차이를 설정으로 분리해 공급사별 코드 복제를 줄였습니다.	특정 공급사만의 복잡한 알고리즘이 많다면 전용 adapter 코드가 더 명확할 수 있습니다.
OFFSET vs Cursor pagination	LIMIT/OFFSET	현재 데이터 규모에서 단순하고 페이지 번호 UI 에 적합하며 병목 근거가 없었습니다.	대규모 데이터의 깊은 페이지 탐색·실시간 변경이 많으면 cursor/keyset 이 유리할 수 있습니다.
Sync vs Async Worker	둘 다 제공	짧은 흐름은 동기가 단순하고, 긴 작업 확장 가능성은 Worker 가 적합해 두 방식을 비교했습니다.	무조건 비동기가 좋은 것이 아니라 실제 작업 시간과 운영 복잡도로 선택합니다.
Docker Compose vs 직접 실행	Docker Compose	PostgreSQL·Redis·FastAPI·Celery 를 같은 설정으로 재현하기 쉽습니다.	서비스 하나만 빠르게 디버깅할 때는 로컬 프로세스를 직접 실행하는 편이 빠를 수 있습니다.

10. 대표 문제 해결 사례 3 개

면접에서는 STAR 보다 더 기술적으로 “문제 → 원인 → 해결 → 검증 → 배운 점” 순서로 말합니다.

사례 1. 동일 CSV 재검수 방지

단계	내용
문제	같은 파일을 다시 올리면 저장 중복은 막아도 검수 CPU 작업은 이미 끝난 뒤였습니다.
원인	중복 여부를 저장 직전에만 확인해 애플리케이션이 검수를 먼저 실행했습니다.
해결	파일 검증·파싱 후 SHA-256 + inspection_version 으로 기존 결과를 사전 조회하고, 없을 때만 검수합니다. 경쟁 요청은 PostgreSQL partial unique index 가 최종 차단합니다.
검증	중복 API 테스트와 경쟁 저장 테스트로 “불필요한 실행 감소”와 “DB 중복 방지”를 별도로 확인했습니다.
배운 점	성능 최적화와 데이터 무결성은 같은 문제가 아니며 각각 다른 안전장치가 필요했습니다.

사례 2. Promotion stale 반영 방지

단계	내용
문제	사용자가 Preview 를 확인한 뒤 staging/운영 데이터가 바뀌면 승인한 내용과 실제 반영 내용이 달라질 수 있습니다.
원인	Preview 와 실행 사이에 시간이 있고, 실행 시점의 DB 상태가 변경될 수 있습니다.
해결	canonical Preview 의 SHA-256 을 expected_preview_hash 로 전달하고 실행 직전 서버가 다시 계산합니다. 다르면 409 preview_stale 로 차단하고, 같을 때만 transaction 으로 반영합니다.
검증	stale, 동시성, rollback, Audit/History, Chromium E2E 와 최종 PostgreSQL 상태를 확인했습니다.
배운 점	사용자 승인 기능은 checkbox 하나가 아니라 승인한 데이터의 동일성까지 보장해야 의미가 있습니다.

사례 3. Docker build 성공 후 runtime import 실패

단계	내용
문제	Dockerfile.aws image build 는 성공했지만 기본 CMD 실행 시 ModuleNotFoundError 로 FastAPI 가 뜨지 않았습니다.
원인	services/와 workers/ 디렉터리가 운영 image 에 포함되지 않았습니다.
해결	필수 package COPY 를 보완하고 .dockerignore/build context 를 정리했습니다. 이후 CI 에 image import, UID 10001, migration, 기본 CMD, /health 검증을 추가했습니다.
검증	GitHub Actions Run 30737512739 에서 AWS runtime smoke 와 기존 test/browser-e2e 가 모두 성공했습니다.
배운 점	build 통과는 실행 가능성을 보장하지 않으므로 실제 runtime 을 최소한 한 번은 자동 검증해야 합니다.

11. 압박 질문 대응

방어적으로 말하지 않고 현재 범위·근거·한계를 먼저 인정한 뒤 기술적 판단을 설명합니다.

압박 1. “인증도 없는데 실서비스라고 할 수 있나요?”

실서비스라고 과장하지 않습니다. 취업 포트폴리오용 MVP 이고 인증·권한·rate limit 은 의도적으로 제외했습니다. 대신 데이터 무결성, transaction, Audit, CI 같은 백엔드 핵심을 먼저 검증했습니다. 실제 외부 사용자를 받는 단계라면 인증·권한과 업로드 정책을 가장 먼저 추가하겠습니다.

압박 2. “Streamlit 이면 백엔드 프로젝트라고 보기 어렵지 않나요?”

Streamlit 은 데모 UI 일 뿐 핵심 검수·DB·Promotion 로직은 FastAPI Service 와 PostgreSQL 에 있습니다. UI 는 CatalogGuardApiClient 로 API 만 호출하고 DB 를 직접 수정하지 않습니다. 그래서 UI 를 다른 프론트엔드로 바꿔도 백엔드 계약은 유지됩니다.

압박 3. “Celery 까지 쓸 정도로 실제 작업이 오래 걸리나요?”

현재 합성 데이터에서 항상 긴 작업이라고 주장하지 않습니다. 동기 API 도 유지하고, 데이터가 커질 때 요청 수명과 작업 실행을 분리하는 구조를 실제로 검증하기 위해 비동기 경로를 추가했습니다. 운영 복잡도가 더 큰 만큼 모든 작업에 Celery 를 쓰지는 않습니다.

압박 4. “10,000 건인데 성능 최적화라고 할 수 있나요?”

대규모 성능 최적화라고 표현하지 않습니다. 기존 10,000/100,000 행 PostgreSQL benchmark 에서 실행 계획과 중앙값을 확인했고 현재 병목 근거가 없어 index 를 추가하지 않았습니다. 핵심은 측정 없이 최적화하지 않은 판단 과정입니다.

압박 5. “AI 전공인데 왜 AI 모델이 없나요?”

이 프로젝트의 핵심 문제는 구조화 상품 데이터 품질과 안전한 백엔드 처리라 규칙 기반 접근이 설명 가능하고 테스트하기 적합했습니다. AI 를 억지로 넣기보다 문제에 맞는 기술을 선택했습니다. 별도 프로젝트에서는 LangGraph·MCP 경험도 있지만 CatalogGuard Lite 에서는 백엔드·데이터 품질 역량을 명확히 보여주려 했습니다.

압박 6. “JSON Profile 이 복잡해지면 결국 유지보수가 어려운 것 아닌가요?”

맞습니다. 모든 공급사 특수 로직을 JSON 에 넣는 것이 목표는 아닙니다. 컬럼 매핑·기본값처럼 공통 모델로 표현 가능한 차이는 Profile 에 두고, 복잡한 알고리즘이 필요해지면 adapter 나 별도 transform 전략을 코드로 분리하는 편이 낫습니다.

압박 7. “왜 AWS 를 계속 커두지 않았나요?”

포트폴리오 staging 검증 목적을 달성한 뒤 비용을 줄이기 위해 EC2·RDS 를 중지했습니다. 실제 EC2·RDS·TLS·Docker 흐름은 검증했고, 최신 코드의 실행 가능성은 GitHub Actions 에서 Dockerfile.aws runtime smoke 로 자동 확인합니다. 최근에 AWS 에 재배포했다고 과장하지 않습니다.

압박 8. “테스트가 너무 많은 것 아닌가요?”

개수 자체를 목표로 만든 것은 아닙니다. 규칙·API·DB·E2E 처럼 책임별로 테스트가 누적됐고, 중요한 것은 실제 결함을 발견한 뒤 회귀 테스트로 남긴 것입니다. 중복되거나 가치 없는 테스트가 보이면 줄이는 것도 유지보수의 일부라고 생각합니다.

12. 현재 한계와 향후 개선

현재 한계	현재 판단	실무 서비스라면
인증·권한·rate limit 미구현	공개 포트폴리오 MVP 범위에서 제외	사용자/역할 모델, 인증, 권한 검사, 업로드 제한 추가
운영 Redis·Celery 배포 미검증	로컬·CI Async E2E 까지 검증	모니터링, retry 정책, worker autoscaling/장애 대응 검토
외부 공급사 실연동 없음	합성 공급사 2 종으로 구조 검증	계약/개인정보를 고려한 실제 feed 연동과 Profile 운영 정책
streaming·증분 ETL 없음	현재 CSV batch MVP 에 불필요	파일 크기·주기·변경량이 커질 때 chunk/증분 처리 도입
대규모 동시 트래픽 측정 없음	현재 성능 자료는 제한된 합성 benchmark	실제 SLA 와 부하 패턴이 생긴 뒤 부하 테스트·관찰성 확장
AWS 자동 배포/laC 없음	수동 staging 경험 + CI runtime smoke 확보	반복 배포 요구가 생기면 laC/배포 자동화 검토

향후 한 가지를 고르라는 질문

지금은 새 기능을 무조건 추가하기보다 지원을 시작하고 실제 채용공고에서 반복적으로 부족하다고 확인되는 항목을 기존 프로젝트에 적용하겠습니다. 인증·권한이나 운영 비동기 배포가 실제 목표 공고에서 반복되면 그중 하나를 우선 보완하겠습니다.

13. 면접 전날 보는 ★★★★★ 핵심 질문

질문 키워드	10 초 핵심 문장
프로젝트 소개	공급사 CSV 표준화 + 품질 검수 + staging + Preview 승인 Promotion/Audit 을 연결한 FastAPI 백엔드 MVP.
왜 FastAPI?	Python 데이터 처리와 API 를 한 언어로 연결하고 Pydantic 계약·단순한 구조가 프로젝트 규모에 적합.
단일 검수 기준	검수는 FastAPI 에서 1 회 실행, Streamlit 은 결과 표시 → 이중 검수와 기준 불일치 제거.
동일 CSV	SHA-256 + inspection_version 사전 조회로 재검수 생략, partial unique index 로 경쟁 요청 최종 차단.
PostgreSQL	관계·transaction·constraint·migration·실행 계획 검증이 중요한 데이터 구조라 선택.
Transaction	ETL/Promotion 일부 실패 시 데이터가 반만 남지 않도록 전체 rollback.
Redis·Celery	긴 작업을 HTTP 요청 수명에서 분리. Redis broker/상태, Celery Worker, PostgreSQL 결과.
ETL	공급사 CSV → JSON Profile → 공통 변환 → normal/reject/summary → staging.
JSON Profile	공급사별 코드 복제 대신 공통으로 표현 가능한 컬럼 차이를 설정으로 분리.
staging	운영 반영 전 임시 안전 구역. 검증·Preview 후 Promotion.
Promotion Preview	실제 변경 전에 insert/update/no-change 와 변경 전후를 확인.
expected hash	사용자가 승인한 Preview 와 실행 시점 결과가 같은지 SHA-256 으로 재검증.
Audit	운영 상품이 언제 어떤 Run 에서 무엇이 바뀌었는지 이전/새 값을 추적.
Docker runtime 문제	build 성공했지만 services/workers 누락으로 실행 실패 → CI 에 runtime smoke 추가.
CI	PostgreSQL/Redis/migration/pytest/Async E2E/Streamlit/Chromium/AWS image runtime 자동 검증.
테스트 철학	1042 passed 숫자보다 실제 장애를 회귀 테스트로 바꾼 경험이 중요.
성능	기존 10k/100k benchmark 와 EXPLAIN ANALYZE 재검토 → 병목 근거 없어 index 추가 안 함.
한계	인증·권한·rate limit, 운영 비용기 배포, streaming/증분 ETL, 실연동은 현재 범위 밖.

14. 모르는 질문을 받았을 때 대응 방법

1 단계 - 아는 범위부터 말하기

"제가 직접 구현한 범위에서는 A 까지 확인했습니다. B 는 현재 프로젝트에서 사용하지 않아 정확한 동작은 확인이 필요합니다."

2 단계 - 추측을 사실처럼 말하지 않기

모르면 "아마 그럴 것 같습니다"로 채우지 말고, 알고 있는 원리와 확인해야 할 항목을 분리합니다.

3 단계 - 프로젝트와 연결하기

"다만 CatalogGuard Lite 에서는 비슷한 문제를 transaction/constraint 로 해결했습니다."처럼 실제 경험으로 돌아옵니다.

4 단계 - 틀렸음을 알았을 때

"제가 방금 설명한 부분은 정확하지 않았습니다. 정정하면 ...입니다."라고 짧게 바로잡습니다. 잘못된 답을 끝까지 방어하지 않습니다.

면접에서 피할 표현

- "FastAPI 가 가장 빠르기 때문에 선택했습니다." → 근거 없는 절대 표현 피하기
- "production-grade 입니다." → 인증·운영 배포 범위를 고려하면 과장
- "대규모 성능 최적화를 했습니다." → 현재 benchmark 범위를 정확히 말하기
- "최근 AWS 에 재배포했습니다." → 최신은 GitHub Actions Docker runtime smoke
- "테스트가 1042 개라 안정적입니다." → 숫자보다 어떤 위험을 검증했는지 설명

마지막 20 초 마무리 문장

마무리

CatalogGuard Lite 를 만들면서 기능을 추가하는 것보다 실제 오류를 재현하고 원인을 찾은 뒤 테스트와 CI 로 재발을 막는 과정이 중요하다는 것을 배웠습니다. 아직 인증·운영 비동기 배포처럼 보완할 부분은 있지만, 현재 구현 범위와 한계를 정확히 설명할 수 있고 지원하면서 실제 요구에 맞춰 개선해 나가겠습니다.

최신 기준

항목	값
HEAD	edd73ed2c00f04aa7d34c0420865463253471dcb
GitHub Actions	30737512739 · test / browser-e2e success
최근 전체 pytest	1042 passed · 88 skipped · 4 deselected
Chromium E2E	2 개 시나리오 · Promotion 포함
AWS Runtime CI	image build/import · UID 10001 · migration · 기본 CMD Uvicorn · /health 200
성능 자료	기존 PostgreSQL 18 benchmark 재검토 · fresh ETL/Promotion benchmark 는 미수행